

✓ IEMOCAP Emotion Recognition — Modelling Pipeline

Wav2Vec2 + Held-Out Session Evaluation

This notebook implements an **Audio-Only Transformer pipeline** for 3-class Speech Emotion Recognition (SER) on the IEMOCAP corpus.

Component	Choice
Model	<code>Wav2Vec2ForSequenceClassification</code> (<code>facebook/wav2vec2-base</code>)
Evaluation	Single held-out session (Ses05) — fully speaker-independent
Loss	<code>CrossEntropyLoss</code> with class weights balanced on the training set
Primary metric	Macro F1-Score (robust to the ~55 % Negative class imbalance)
Classes	<code>Negative</code> (0) · <code>Positive</code> (1) · <code>Neutral</code> (2)

```
import torch
print("CUDA Available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("Device Name:", torch.cuda.get_device_name(0))
```

```
CUDA Available: True
Device Name: Tesla T4
```

```
import io
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from transformers import Wav2Vec2Processor, Wav2Vec2ForSequenceClassification
from sklearn.model_selection import LeaveOneGroupOut
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import f1_score, classification_report
import soundfile as sf
import torchaudio.transforms as T
from tqdm import tqdm
from transformers import get_linear_schedule_with_warmup
import matplotlib.pyplot as plt
import itertools
import warnings
warnings.filterwarnings('ignore')

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')
print(f'PyTorch version: {torch.__version__}')
```

```
Using device: cuda
PyTorch version: 2.11.0+cu128
```

✓ Step 1: Session Extraction & Label Mapping

Extract the integer session number from the `file` column (`Ses01` → `1`, ..., `Ses05` → `5`) to use as group IDs for LOSO-CV, and map the 3-class `emotion_group` labels to integers.

```
from datasets import load_dataset

# Load IEMOCAP from HuggingFace (mirrors EDA notebook)
ds = load_dataset('AbstractTTS/IEMOCAP')
df = ds['train'].to_pandas()

# Emotion grouping — same mapping as the EDA notebook
emotion_group_map = {
    'happy': 'Positive',
    'excited': 'Positive',
    'angry': 'Negative',
    'frustrated': 'Negative',
    'fear': 'Negative',
    'sad': 'Negative',
    'disgust': 'Negative',
    'other': 'Neutral',
    'neutral': 'Neutral',
    'surprise': 'Neutral',
}

df['emotion_group'] = df['major_emotion'].map(emotion_group_map)
```

```
# Extract integer session number (1-5) for L0S0 grouping
df['session_num'] = df['file'].str.extract(r'Ses\d{2}').astype(int)

# Encode emotion_group -> integer label
LABEL_MAP = {'Negative': 0, 'Positive': 1, 'Neutral': 2}
ID2LABEL = {v: k for k, v in LABEL_MAP.items()}
df['label'] = df['emotion_group'].map(LABEL_MAP)

# Modelling DataFrame: preserve the audio column for the Dataset class
df_model = (
    df[['file', 'audio', 'emotion_group', 'label', 'session_num']]
    .dropna()
    .reset_index(drop=True)
)

print(f'Total records : {len(df_model)}')
print(f'\nLabel distribution:')
print(df_model['emotion_group'].value_counts())
print(f'\nSession distribution:')
print(df_model['session_num'].value_counts().sort_index())
```

```
README.md: 100% 1.07k/1.07k [00:00<00:00, 39.2kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
data/train-00000-of-00003.parquet: 100% 489M/489M [00:03<00:00, 194MB/s]
data/train-00001-of-00003.parquet: 100% 456M/456M [00:02<00:00, 236MB/s]
data/train-00002-of-00003.parquet: 100% 462M/462M [00:03<00:00, 133MB/s]
Generating train split: 100% 10039/10039 [00:07<00:00, 1433.23 examples/s]
Total records : 10039

Label distribution:
emotion_group
Negative    5545
Positive    2632
Neutral     1862
Name: count, dtype: int64

Session distribution:
session_num
1      1819
2      1811
3      2136
4      2103
5      2170
Name: count, dtype: int64
```

```
# === Sum-then-Argmax: corrected ground truth + human-consensus score ===
group_emotions = {
    'Positive': ['happy', 'excited'],
    'Negative': ['angry', 'frustrated', 'fear', 'sad', 'disgust'],
    'Neutral': ['neutral', 'surprise'],
}
for group, cols in group_emotions.items():
    df[f'{group}_prob'] = df[cols].sum(axis=1)

prob_cols = [f'{g}_prob' for g in group_emotions]
df['Corrected_Ground_Truth'] = df[prob_cols].idxmax(axis=1).str.replace('_prob', '', regex=False)
df['consensus'] = df[prob_cols].max(axis=1)
df['corrected_label'] = df['Corrected_Ground_Truth'].map(LABEL_MAP)

# df_model only kept a subset of df's columns – merge the new ones in by file id
df_model = df_model.merge(
    df[['file', 'Corrected_Ground_Truth', 'consensus', 'corrected_label']],
    on='file', how='left',
)

print("Corrected_Ground_Truth distribution:")
print(df['Corrected_Ground_Truth'].value_counts().to_string())
print(f"\nMean human consensus: {df['consensus'].mean():.4f}")

n_flipped = (df['emotion_group'] != df['Corrected_Ground_Truth']).sum()
print(f"Labels flipped vs. major_emotion grouping: {n_flipped} / {len(df)} ({n_flipped/len(df)*100:.2f}%)")
```

```
Corrected_Ground_Truth distribution:
Corrected_Ground_Truth
Negative    5721
Positive    2504
```

Neutral 1814

Mean human consensus: 0.8055

Labels flipped vs. major_emotion grouping: 503 / 10039 (5.01%)

```
# — Audio Duration Analysis —————
# Run once to determine the right MAX_DURATION; then update the value below.
import soundfile as sf

def _get_dur(audio_data):
    raw = audio_data.get('bytes') if isinstance(audio_data, dict) else None
    if raw is None:
        return None
    with sf.SoundFile(io.BytesIO(raw)) as f:
        return len(f) / f.samplerate

print('Computing audio durations... (may take 2–3 minutes)')
_dur = df_model['audio'].apply(_get_dur).dropna()

print(f'\nAudio Duration Statistics (seconds):')
print(f'  Count : {len(_dur):,}')
print(f'  Mean  : {_dur.mean():.2f}s')
print(f'  Median: {_dur.median():.2f}s')
print(f'  Std   : {_dur.std():.2f}s')
print(f'  Min   : {_dur.min():.2f}s | Max: {_dur.max():.2f}s')
print(f'\n  Percentiles:')
for _p in [50, 75, 90, 95, 99]:
    print(f'    {_p:3d}th : {_dur.quantile(_p / 100):.2f}s')

_p90 = _dur.quantile(0.90)
print(f'\n→ Recommended MAX_DURATION (90th pct): {_p90:.1f}s')
print('  Update MAX_DURATION in the Hyperparameters cell to this value.')

fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(_dur, bins=60, color='steelblue', edgecolor='white', alpha=0.85)
ax.axvline(_dur.median(), color='orange', linestyle='--', linewidth=1.5,
           label=f'Median: {_dur.median():.1f}s')
ax.axvline(_p90, color='crimson', linestyle='--', linewidth=1.5,
           label=f'90th pct: {_p90:.1f}s')
ax.set_xlabel('Duration (s)'); ax.set_ylabel('Count')
ax.set_title('IEMOCAP Utterance Duration Distribution')
ax.legend(); plt.tight_layout(); plt.show()
```

Computing audio durations... (may take 2–3 minutes)

Audio Duration Statistics (seconds):

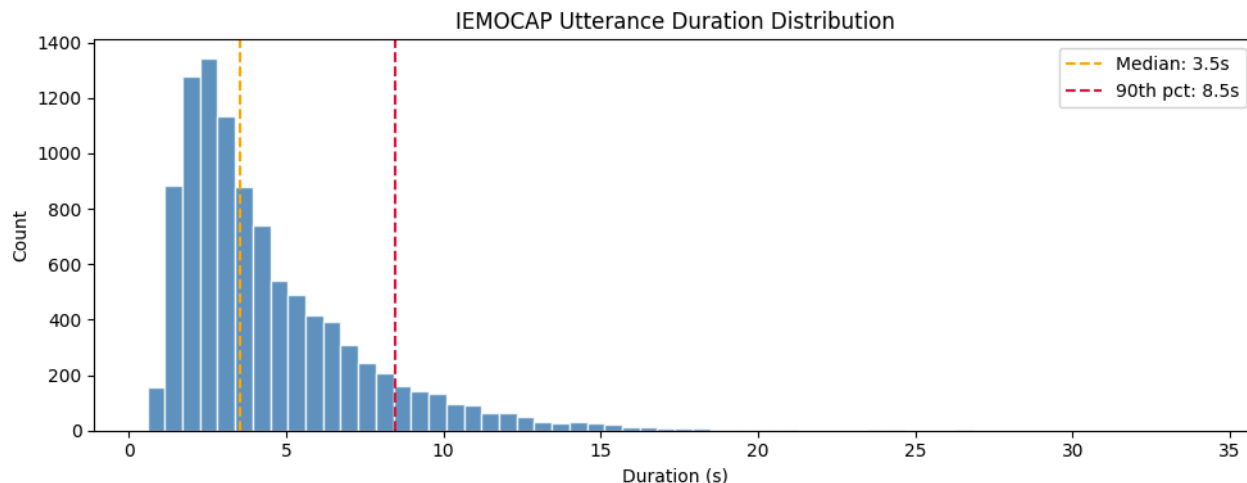
Count : 10,039
 Mean : 4.46s
 Median : 3.52s
 Std : 3.06s
 Min : 0.58s | Max: 34.14s

Percentiles:

50th : 3.52s
 75th : 5.71s
 90th : 8.45s
 95th : 10.53s
 99th : 15.08s

→ Recommended MAX_DURATION (90th pct): 8.5s

Update MAX_DURATION in the Hyperparameters cell to this value.



Step 2: Custom PyTorch Dataset & Audio Preprocessing

`IEMOCAPDataset` handles four responsibilities:

1. **Loading** raw audio from the HuggingFace `bytes` dict via `soundfile`.
2. **Mono conversion** — any stereo recordings are averaged to one channel.
3. **Resampling** to 16 000 Hz using `torchaudio.transforms.Resample` (Wav2Vec2 requires exactly 16 kHz).
4. **Feature extraction** via `Wav2Vec2Processor`: zero-mean unit-variance normalisation plus padding / truncation to a fixed `max_duration`.

No hand-crafted MFCC or spectrogram features are needed — the processor works directly on the raw float waveform.

```
CHECKPOINT = 'facebook/wav2vec2-base'
TARGET_SR = 16_000 # Hz — required by all Wav2Vec2 variants

class IEMOCAPDataset(Dataset):
    # PyTorch Dataset for IEMOCAP audio emotion classification.
    # Each __getitem__ returns a dict with input_values, attention_mask, and labels.

    def __init__(self, df, processor, max_duration=8.5, target_sr=TARGET_SR):
        self.df = df.reset_index(drop=True)
        self.processor = processor
        self.max_length = int(max_duration * target_sr)
        self.target_sr = target_sr

    def __len__(self):
        return len(self.df)

    def _load_audio(self, audio_data):
        # Return (waveform float32 ndarray, sampling_rate) from a HuggingFace audio dict.
        if isinstance(audio_data, dict):
            raw_bytes = audio_data.get('bytes')
            if raw_bytes is None:
                raise ValueError('Audio dict is missing the bytes key.')
            waveform, sr = sf.read(io.BytesIO(raw_bytes), dtype='float32')
        elif isinstance(audio_data, np.ndarray):
            waveform = audio_data.astype(np.float32)
            sr = self.target_sr
        else:
            raise TypeError(f'Unsupported audio type: {type(audio_data)}')

        # Stereo -> mono by averaging channels
        if waveform.ndim > 1:
            waveform = waveform.mean(axis=-1)

        return waveform, sr

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        waveform, sr = self._load_audio(row['audio'])

        # Resample to TARGET_SR if the stored rate differs
        if sr != self.target_sr:
            waveform_t = torch.from_numpy(waveform).unsqueeze(0)
            resampler = T.Resample(orig_freq=sr, new_freq=self.target_sr)
            waveform = resampler(waveform_t).squeeze(0).numpy()

        # Normalise + pad/truncate to a fixed sequence length
        encoded = self.processor(
            waveform,
            sampling_rate=self.target_sr,
            return_tensors='pt',
            max_length=self.max_length,
            truncation=True,
            padding='max_length',
        )

        return {
            'input_values' : encoded['input_values'].squeeze(0),          # (max_length,)
            'attention_mask': encoded.get(
                'attention_mask', torch.ones(self.max_length)
            ).squeeze(0),          # (max_length,)
            'labels': torch.tensor(row['label'], dtype=torch.long),      # scalar
        }

    # Quick sanity check — verify tensor shapes before training
    _processor = Wav2Vec2Processor.from_pretrained(CHECKPOINT)
```

```
_check_ds = IEMOCAPDataset(df_model.head(2), _processor)
_sample   = _check_ds[0]

print('input_values shape:', _sample['input_values'].shape) # expect (128000,)
print('attention_mask shape:', _sample['attention_mask'].shape) # expect (128000,)
print('label      :', _sample['labels'].item(),
      '->', ID2LABEL[_sample['labels'].item()])

del _check_ds, _sample
```

```
preprocessor_config.json: 100%          159/159 [00:00<00:00, 13.8kB/s]
config.json: 100%                    1.84k/1.84k [00:00<00:00, 194kB/s]
tokenizer_config.json: 100%          163/163 [00:00<00:00, 16.2kB/s]
vocab.json: 100%                     291/291 [00:00<00:00, 34.8kB/s]
special_tokens_map.json: 100%        85.0/85.0 [00:00<00:00, 11.6kB/s]
input_values shape: torch.Size([136000])
attention_mask shape: torch.Size([136000])
label      : 2 -> Neutral
```

```
# — Hyperparameters —————
CHECKPOINT = 'facebook/wav2vec2-base'
NUM_LABELS = 3
BATCH_SIZE = 8      # reduce to 4 if GPU runs out of memory
NUM_EPOCHS = 8
LEARNING_RATE = 2e-5
MAX_DURATION = 8.5  # seconds – 90th percentile of IEMOCAP utterance durations
PATIENCE = 3        # early stopping patience (epochs without improvement)
NUM_WORKERS = 2
TEST_SESSION = 5    # IEMOCAP session held out as the test set (1–5)
```

Step 3: Train / Test Split

The dataset is split into a **training set** (Sessions 1–4, ~8,000 utterances) and a **held-out test set** (Session 5, ~2,000 utterances).

Using a full session as the test set guarantees **complete speaker independence** — the model never encounters any utterance from the test speakers during training, avoiding *Speaker Leakage*.

Split	Sessions	Approx. size
Train	1, 2, 3, 4	~8,000 utterances
Test	5 (held out)	~2,000 utterances

```
train_df = df_model[df_model['session_num'] != TEST_SESSION].reset_index(drop=True)
test_df = df_model[df_model['session_num'] == TEST_SESSION].reset_index(drop=True)

train_sessions = sorted(s for s in df_model['session_num'].unique() if s != TEST_SESSION)
session_str = ', '.join(str(s) for s in train_sessions)

print(f'Train set : {len(train_df):,} samples (Sessions {session_str})')
print(f'Test set : {len(test_df):,} samples (Session {TEST_SESSION} – held out)')
print(f'\nTrain label distribution:')
print(train_df['emotion_group'].value_counts().to_string())
print(f'\nTest label distribution:')
print(test_df['emotion_group'].value_counts().to_string())
```

```
Train set : 7,869 samples (Sessions 1, 2, 3, 4)
Test set : 2,170 samples (Session 5 – held out)
```

```
Train label distribution:
emotion_group
Negative    4396
Positive   2019
Neutral    1454
```

```
Test label distribution:
emotion_group
Negative    1149
Positive    613
Neutral     408
```

Steps 4 & 5: Class Weights · Model Initialisation · Training

Step 4 — Class weights (from training set): Class weights are computed from the training set using

`sklearn.utils.class_weight.compute_class_weight('balanced', ...)`, which implements:

$$weight_c = \frac{N}{C \cdot n_c}$$

where N = total training samples, C = number of classes, n_c = samples in class c .

These weights are passed to `nn.CrossEntropyLoss(weight=...)`, so the model incurs a proportionally higher penalty for misclassifying the minority Positive / Neutral classes. Computing weights **from the training set only** ensures no information from the held-out test session leaks into training.

Step 5 — Model + Training: `Wav2Vec2ForSequenceClassification.from_pretrained(...)` loads the pretrained encoder and attaches a fresh 3-way classification head. The convolutional feature encoder is frozen via `model.freeze_feature_encoder()` to preserve low-level audio representations and prevent catastrophic forgetting at the start of fine-tuning. A linear warmup + linear decay scheduler and gradient clipping (`max_norm=1.0`) stabilise training. Early stopping (patience = 3) halts training when test F1 stops improving.

```
def train_epoch(model, loader, optimiser, loss_fn, device, scheduler=None):
    model.train()
    total_loss, all_preds, all_labels = 0.0, [], []

    for batch in tqdm(loader, desc=' train', leave=False):
        input_values = batch['input_values'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        optimiser.zero_grad()
        logits = model(input_values=input_values, attention_mask=attention_mask).logits
        loss = loss_fn(logits, labels)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimiser.step()
        if scheduler is not None:
            scheduler.step()

        total_loss += loss.item()
        all_preds.extend(torch.argmax(logits, dim=-1).cpu().tolist())
        all_labels.extend(labels.cpu().tolist())

    avg_loss = total_loss / len(loader)
    macro_f1 = f1_score(all_labels, all_preds, average='macro', zero_division=0)
    return avg_loss, macro_f1

def eval_epoch(model, loader, loss_fn, device):
    model.eval()
    total_loss, all_preds, all_labels = 0.0, [], []

    with torch.no_grad():
        for batch in tqdm(loader, desc=' val ', leave=False):
            input_values = batch['input_values'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['labels'].to(device)

            logits = model(input_values=input_values, attention_mask=attention_mask).logits
            loss = loss_fn(logits, labels)

            total_loss += loss.item()
            all_preds.extend(torch.argmax(logits, dim=-1).cpu().tolist())
            all_labels.extend(labels.cpu().tolist())

    avg_loss = total_loss / len(loader)
    macro_f1 = f1_score(all_labels, all_preds, average='macro', zero_division=0)
    return avg_loss, macro_f1

def eval_epoch_with_preds(model, loader, loss_fn, device):
    model.eval()
    total_loss, all_preds, all_labels = 0.0, [], []

    with torch.no_grad():
        for batch in tqdm(loader, desc=' eval ', leave=False):
            input_values = batch['input_values'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['labels'].to(device)

            logits = model(input_values=input_values, attention_mask=attention_mask).logits
            loss = loss_fn(logits, labels)

            total_loss += loss.item()
            all_preds.extend(torch.argmax(logits, dim=-1).cpu().tolist())
            all_labels.extend(labels.cpu().tolist())
```

```

avg_loss = total_loss / len(loader)
macro_f1 = f1_score(all_labels, all_preds, average='macro', zero_division=0)
return avg_loss, macro_f1, all_preds, all_labels

```

```

sep = '=' * 60
print(f'\n{sep}')
print(f'Training | Held-out test session: Ses0{TEST_SESSION}')
print(f' Train: {len(train_df):,} samples | Test: {len(test_df):,} samples')

# — Step 4: class weights from training set —————
train_labels = train_df['label'].values
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.array([0, 1, 2]),
    y=train_labels
)
weight_tensor = torch.tensor(class_weights, dtype=torch.float32).to(device)
loss_fn = nn.CrossEntropyLoss(weight=weight_tensor)

print(f' Class weights -> '
      f'Negative: {class_weights[0]:.3f} | '
      f'Positive: {class_weights[1]:.3f} | '
      f'Neutral : {class_weights[2]:.3f}')

# — Datasets & DataLoaders —————
processor = Wav2Vec2Processor.from_pretrained(CHECKPOINT)
train_dataset = IEMOCAPDataset(train_df, processor, max_duration=MAX_DURATION)
test_dataset = IEMOCAPDataset(test_df, processor, max_duration=MAX_DURATION)

train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE,
    shuffle=True, num_workers=NUM_WORKERS, pin_memory=True
)
test_loader = DataLoader(
    test_dataset, batch_size=BATCH_SIZE,
    shuffle=False, num_workers=NUM_WORKERS, pin_memory=True
)

# — Step 5: model initialisation + freeze feature encoder —————
model = Wav2Vec2ForSequenceClassification.from_pretrained(
    CHECKPOINT,
    num_labels=NUM_LABELS,
    id2label=ID2LABEL,
    label2id=LABEL_MAP,
    ignore_mismatched_sizes=True,
).to(device)

model.freeze_feature_encoder()
print(' Feature encoder frozen.')

optimiser = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
total_steps = len(train_loader) * NUM_EPOCHS
warmup_steps = max(1, int(0.1 * total_steps))
scheduler = get_linear_schedule_with_warmup(optimiser, warmup_steps, total_steps)

best_val_f1 = 0.0
best_preds = []
best_labels_list = []
epochs_no_improve = 0

# — Training loop —————
for epoch in range(NUM_EPOCHS):
    tr_loss, tr_f1 = train_epoch(model, train_loader, optimiser, loss_fn, device, scheduler)
    vl_loss, vl_f1 = eval_epoch(model, test_loader, loss_fn, device)

    print(f' Ep {epoch + 1}/{NUM_EPOCHS} | '
          f'Train loss: {tr_loss:.4f} F1: {tr_f1:.4f} | '
          f'Test loss: {vl_loss:.4f} F1: {vl_f1:.4f}')

    if vl_f1 > best_val_f1:
        best_val_f1 = vl_f1
        torch.save(model.state_dict(), 'best_model.pt')
        _, _, best_preds, best_labels_list = eval_epoch_with_preds(
            model, test_loader, loss_fn, device
        )
        print(f' >> New best saved (Test Macro F1 = {best_val_f1:.4f})')
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= PATIENCE:
            print(f' Early stopping triggered at epoch {epoch + 1}')

```

```

        break

fold_results = [{
    'test_session': f'Ses0{TEST_SESSION}',
    'best_val_f1' : round(best_val_f1, 4),
    'preds'      : best_preds,
    'labels'     : best_labels_list,
}]

del model, train_dataset, test_dataset, train_loader, test_loader
torch.cuda.empty_cache()

```

```

=====
Training | Held-out test session: Ses05
  Train: 7,869 samples | Test: 2,170 samples
  Class weights -> Negative: 0.597 | Positive: 1.299 | Neutral : 1.804

pytorch_model.bin: 100%                               380M/380M [00:01<00:00, 243MB/s]

Loading weights: 100%                                211/211 [00:00<00:00, 7138.46it/s]

[transformers] Wav2Vec2ForSequenceClassification LOAD REPORT from: facebook/wav2vec2-base

```

Key	Status
quantizer.weight_proj.bias	UNEXPECTED
project_q.weight	UNEXPECTED
project_hid.weight	UNEXPECTED
quantizer.weight_proj.weight	UNEXPECTED
project_hid.bias	UNEXPECTED
project_q.bias	UNEXPECTED
quantizer.codevectors	UNEXPECTED
projector.bias	MISSING
classifier.weight	MISSING
projector.weight	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your own data.

Feature encoder frozen.

train: 0% | 0/984 [00:00<?, ?it/s]

model.safetensors: 100% 380M/380M [00:05<00:00, 34.1MB/s]

Ep 1/8 | Train loss: 1.0475 F1: 0.4246 | Test loss: 0.9394 F1: 0.5278

>> New best saved (Test Macro F1 = 0.5278)

Ep 2/8 | Train loss: 0.8825 F1: 0.5836 | Test loss: 0.8271 F1: 0.5969

>> New best saved (Test Macro F1 = 0.5969)

Ep 3/8 | Train loss: 0.7145 F1: 0.6914 | Test loss: 0.9126 F1: 0.6070

>> New best saved (Test Macro F1 = 0.6070)

Ep 4/8 | Train loss: 0.5613 F1: 0.7785 | Test loss: 0.8681 F1: 0.6398

>> New best saved (Test Macro F1 = 0.6398)

Ep 5/8 | Train loss: 0.4657 F1: 0.8386 | Test loss: 1.1833 F1: 0.6179

Ep 6/8 | Train loss: 0.3653 F1: 0.8899 | Test loss: 1.3743 F1: 0.6209

Ep 7/8 | Train loss: 0.2980 F1: 0.9171 | Test loss: 1.3743 F1: 0.6209

Early stopping triggered at epoch 7

```

# — Results Summary —
result = fold_results[0]

print('\n' + '=' * 60)
print('RESULTS — Held-out Session')
print('=' * 60)
print(f'  Test session   : {result["test_session"]}')
print(f'  Best Macro F1   : {result["best_val_f1"]:.4f}')
print(f'\nModel checkpoint saved to: best_model.pt')

```

```

=====
RESULTS — Held-out Session
=====

```

```

  Test session   : Ses05
  Best Macro F1   : 0.6398

```

Model checkpoint saved to: best_model.pt

Results

Confusion matrix and per-class F1 breakdown for the held-out test session.

```

import itertools
from sklearn.metrics import confusion_matrix, classification_report

```

```

all_preds = fold_results[0]['preds']
all_labels = fold_results[0]['labels']
class_names = ['Negative', 'Positive', 'Neutral']

# — Confusion matrix —————
cm = confusion_matrix(all_labels, all_preds)
cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

im0 = axes[0].imshow(cm, interpolation='nearest', cmap='Blues')
axes[0].set_title('Confusion Matrix – Raw Counts', fontsize=13, weight='bold')
plt.colorbar(im0, ax=axes[0])
for i, j in itertools.product(range(3), range(3)):
    axes[0].text(j, i, str(cm[i, j]), ha='center', va='center', fontsize=12)
axes[0].set_xticks(range(3)); axes[0].set_xticklabels(class_names)
axes[0].set_yticks(range(3)); axes[0].set_yticklabels(class_names)
axes[0].set_xlabel('Predicted'); axes[0].set_ylabel('True')

im1 = axes[1].imshow(cm_norm, interpolation='nearest', cmap='Blues', vmin=0, vmax=1)
axes[1].set_title('Confusion Matrix – Normalised', fontsize=13, weight='bold')
plt.colorbar(im1, ax=axes[1])
for i, j in itertools.product(range(3), range(3)):
    axes[1].text(j, i, f'{cm_norm[i, j]:.2f}', ha='center', va='center', fontsize=12)
axes[1].set_xticks(range(3)); axes[1].set_xticklabels(class_names)
axes[1].set_yticks(range(3)); axes[1].set_yticklabels(class_names)
axes[1].set_xlabel('Predicted'); axes[1].set_ylabel('True')

plt.suptitle(f'Test Session: {fold_results[0]["test_session"]} | '
             f'Macro F1: {fold_results[0]["best_val_f1"]:.4f}',
             fontsize=13, weight='bold')
plt.tight_layout()
plt.show()

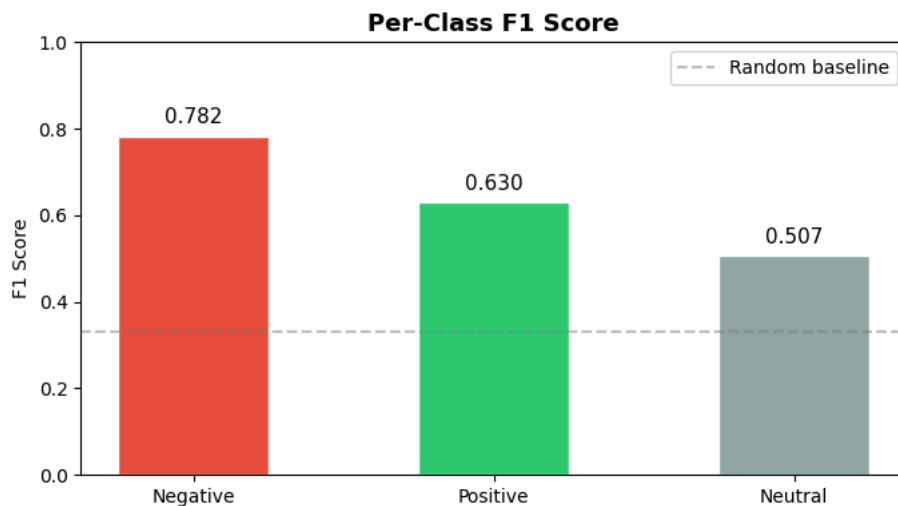
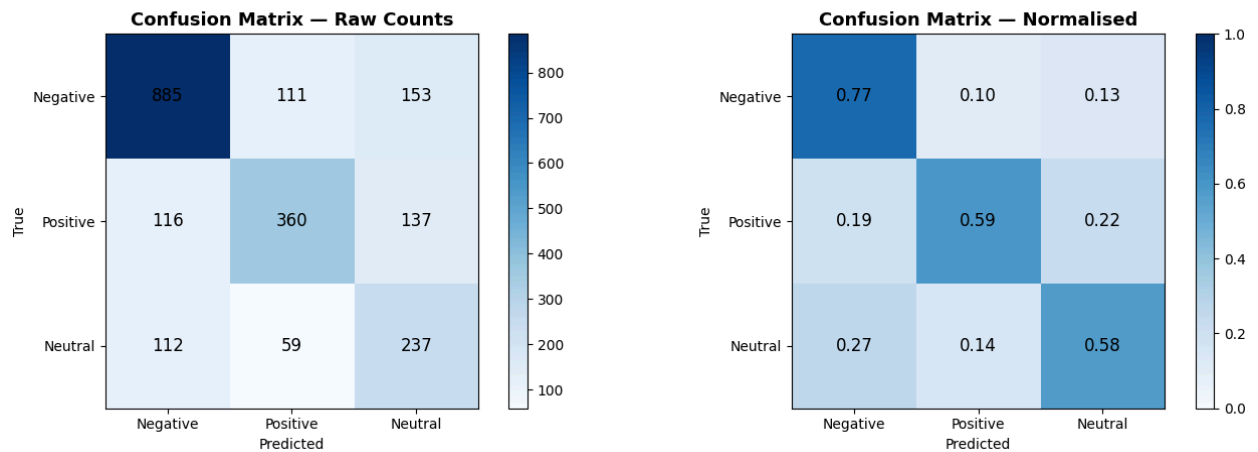
# — Per-class F1 bar chart —————
report = classification_report(all_labels, all_preds,
                              target_names=class_names, output_dict=True, zero_division=0)
class_f1 = [report[cls]['f1-score'] for cls in class_names]
colours = ['#e74c3c', '#2ecc71', '#95a5a6']

fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(class_names, class_f1, color=colours, edgecolor='white', width=0.5)
ax.set_ylim(0, 1)
ax.axhline(y=0.33, color='gray', linestyle='--', alpha=0.5, label='Random baseline')
ax.set_title('Per-Class F1 Score', fontsize=13, weight='bold')
ax.set_ylabel('F1 Score')
for bar, val in zip(bars, class_f1):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.02,
            f'{val:.3f}', ha='center', va='bottom', fontsize=11)
ax.legend(); plt.tight_layout(); plt.show()

# — Full classification report —————
print(f'Classification Report – {fold_results[0]["test_session"]}')
print(classification_report(all_labels, all_preds,
                           target_names=class_names, zero_division=0))

```

Test Session: Ses05 | Macro F1: 0.6398



Classification Report — Ses05

	precision	recall	f1-score	support
Negative	0.80	0.77	0.78	1149
Positive	0.68	0.59	0.63	613
Neutral	0.45	0.58	0.51	408
accuracy			0.68	2170
macro avg	0.64	0.65	0.64	2170
weighted avg	0.70	0.68	0.69	2170

✓ Corrected Ground-Truth Evaluation (Sum-then-Argmax)

Re-evaluates the **already-trained** checkpoint against a ground truth built by summing the 9 raw soft-label probabilities into Negative/Positive/Neutral *before* taking the argmax (rather than grouping the `major_emotion` hard label after the fact). No retraining — this reloads `audio_best_model.pt` (the Drive checkpoint also used by `late_fusion.ipynb`) and runs a forward pass only. Also reports the Consensus Tier Table: accuracy/F1 stratified by how much annotators agreed on each utterance's grouped label.

```
from sklearn.metrics import accuracy_score

def corrected_eval_report(test_df, y_pred, model_name="Model"):
    """Compare old vs. corrected Macro F1, and break down performance by human-consensus tier."""
    y_pred = np.array(y_pred)
    y_true_old = test_df['label'].values
    y_true_corrected = test_df['corrected_label'].values

    old_f1 = f1_score(y_true_old, y_pred, average='macro')
    new_f1 = f1_score(y_true_corrected, y_pred, average='macro')
    print(f"=== {model_name} ===")
    print(f"Old Macro F1      (major_emotion GT)    : {old_f1:.4f}")
```

```

print(f"Corrected Macro F1 (Sum-then-Argmax GT) : {new_f1:.4f}  ( $\Delta$  {new_f1 - old_f1:+.4f})")

bins = [0, 0.5, 0.75, 1.0]
tier_labels = ['Low (<0.5)', 'Medium (0.5-0.75)', 'High (>0.75)']
tiers = pd.cut(test_df['consensus'], bins=bins, labels=tier_labels, include_lowest=True)

rows = []
for tier in tier_labels:
    mask = (tiers == tier).values
    if mask.sum() == 0:
        continue
    rows.append({
        'Tier': tier, 'N': int(mask.sum()),
        'Accuracy': accuracy_score(y_true_corrected[mask], y_pred[mask]),
        'Macro F1': f1_score(y_true_corrected[mask], y_pred[mask], average='macro'),
    })
tier_table = pd.DataFrame(rows)
print(tier_table.to_string(index=False))
print()
return old_f1, new_f1, tier_table

```

```

# No retraining: reload the Drive checkpoint fresh and run a forward pass only.
# (train_loader/test_loader/model were deleted after the training cell above, so we
# rebuild the lightweight test_loader from test_df and reload weights from Drive.)
import os
import sys

IN_COLAB = "google.colab" in sys.modules
if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')
    CHECKPOINT_DIR = "/content/drive/MyDrive"
else:
    CHECKPOINT_DIR = os.path.expanduser("~/Downloads")

AUDIO_CKPT_PATH = os.path.join(CHECKPOINT_DIR, "best_model.pt") # same Drive file late_fusion.ipynb uses

test_dataset = IEMOCAPDataset(test_df, processor, max_duration=MAX_DURATION)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
                          num_workers=NUM_WORKERS, pin_memory=True)

eval_model = Wav2Vec2ForSequenceClassification.from_pretrained(
    CHECKPOINT, num_labels=NUM_LABELS, id2label=ID2LABEL, label2id=LABEL_MAP,
    ignore_mismatched_sizes=True,
).to(device)
eval_model.load_state_dict(torch.load(AUDIO_CKPT_PATH, map_location=device))
eval_model.freeze_feature_encoder()
eval_model.eval()

all_preds = []
with torch.no_grad():
    for batch in tqdm(test_loader, desc='eval'):
        logits = eval_model(
            input_values=batch['input_values'].to(device),
            attention_mask=batch['attention_mask'].to(device),
        ).logits
        all_preds.extend(torch.argmax(logits, dim=-1).cpu().tolist())

corrected_eval_report(test_df, all_preds, "Audio-only")

```